PYTHON FUNDAMENTALS • PRACTICE WORKBOOK

String Methods

50 focused programming problems

Practice these five methods

endswith()	**capitalize()**	**replace()**
find()	**count()**	and their edge cases

WHAT YOU WILL PRACTISE

Choose the right string method, reason about its exact behavior, test unusual inputs, and combine methods to solve small text-processing tasks. The problems progress from single-method exercises to practical scenarios.

15	15	15	5
Easy	Medium	Hard	Real-world / tricky

HOW TO USE THIS WORKBOOK

1. Solve each question in VS Code without copying a solution.
2. Check the example, then add at least one extra test of your own.
3. Keep each solution in a clearly named .py file and commit your work to GitHub.
4. Mark a question complete only when you can explain why the result is correct.

Important: The methods below are string methods, called on a string such as `text.endswith(".py")`. This workbook intentionally does not include full solutions.

Quick Reference

Read this page before you start. The questions test the behavior below, including edge cases that are easy to miss.

endswith(suffix)
Returns True if the string ends with the specified suffix. A tuple can contain several acceptable suffixes. Matching is case-sensitive.
`"report.pdf".endswith(".pdf") -> True`

capitalize()
Returns a new string with its first character capitalized and the remaining characters lowercased. It does not capitalize every word.
`"pYTHON".capitalize() -> "Python"`

replace(old, new[, count])
Returns a new string with occurrences of old replaced by new. By default, all are replaced; count can limit the number replaced.
`"a-a-a".replace("-", " ", 1) -> "a a-a"`

find(sub[, start[, end]])
Returns the lowest index of the first match in the search range. Returns -1 when no match is found. Indexing starts at 0.
`"banana".find("na") -> 2`

count(sub[, start[, end]])
Counts non-overlapping occurrences of a substring. It is case-sensitive and counts substrings, not necessarily whole words.
`"aaaaa".count("aa") -> 2`

Always remember: strings are immutable. A method such as replace() returns a new string; it does not silently change the original variable. Search methods are case-sensitive unless you explicitly transform the text first.

Reference: Python documentation, Built-in Types - String Methods: docs.python.org/3/library/stdtypes.html#string-methods

PRACTICE SECTION

EASY

Build confidence with one method at a time. Focus on exact output and case sensitivity.

E01 Check a Python file extension

FOCUS: endswith()

Given a filename, determine whether it ends with the exact suffix ".py". Print a Boolean result. The check is case-sensitive.

TEST CASE

```
filename = "calculator.py"
```

EXPECTED RESULT

```
True
```

MY PLAN / NOTES

E02 Check an image extension

FOCUS: endswith()

Check whether the filename ends with ".png". Do not inspect any part of the name before the final suffix.

TEST CASE

```
filename = "profile.png"
```

EXPECTED RESULT

```
True
```

MY PLAN / NOTES

E03 Capitalize a first name

FOCUS: capitalize()

A first name was entered in lowercase. Return the result of capitalizing it using the specified string method.

TEST CASE

```
name = "riya"
```

EXPECTED RESULT

```
Riya
```

MY PLAN / NOTES

PRACTICE SECTION

EASY

Build confidence with one method at a time. Focus on exact output and case sensitivity.

E04 Understand capitalize() exactly

FOCUS: capitalize()

Apply capitalize() to the given text. Pay attention to what happens to letters after the first character.

TEST CASE
```
text = "pYTHON"
```

EXPECTED RESULT
```
Python
```

MY PLAN / NOTES

E05 Replace one word

FOCUS: replace()

Replace every occurrence of the word "cat" with "dog" in the provided sentence.

TEST CASE
```
text = "the cat chased the cat"
```

EXPECTED RESULT
```
the dog chased the dog
```

MY PLAN / NOTES

E06 Replace spaces with hyphens

FOCUS: replace()

Create a new string in which every ordinary space is replaced with a hyphen. Keep all other characters unchanged.

TEST CASE
```
text = "learn python daily"
```

EXPECTED RESULT
```
learn-python-daily
```

MY PLAN / NOTES

PRACTICE SECTION

EASY

Build confidence with one method at a time. Focus on exact output and case sensitivity.

E07 Find a substring

FOCUS: find()

Find the index of the first occurrence of the substring "code". Remember that string indexes begin at 0.

TEST CASE

```
text = "learn to code"
```

EXPECTED RESULT

```
9
```

MY PLAN / NOTES

E08 Handle a missing substring

FOCUS: find()

Search for the substring "Java". Print exactly the value returned by find() when the substring is absent.

TEST CASE

```
text = "I am learning Python"
```

EXPECTED RESULT

```
-1
```

MY PLAN / NOTES

E09 Count a character

FOCUS: count()

Count how many times the lowercase character "a" appears in the string.

TEST CASE

```
text = "banana"
```

EXPECTED RESULT

```
3
```

MY PLAN / NOTES

PRACTICE SECTION

EASY

Build confidence with one method at a time. Focus on exact output and case sensitivity.

E10 Count a short substring

FOCUS: count()

Count the occurrences of the exact, case-sensitive substring "is" in the sentence.

TEST CASE

```
text = "This is it"
```

EXPECTED RESULT

```
2
```

MY PLAN / NOTES

E11 Accept either of two suffixes

FOCUS: endswith()

Check whether the filename ends with either ".jpg" or ".png". Use the tuple form supported by endswith().

TEST CASE

```
filename = "photo.jpg"
```

EXPECTED RESULT

```
True
```

MY PLAN / NOTES

E12 Update a year in text

FOCUS: replace()

Replace every occurrence of "2025" with "2026" in the announcement.

TEST CASE

```
text = "Events in 2025: review 2025 goals"
```

EXPECTED RESULT

```
Events in 2026: review 2026 goals
```

MY PLAN / NOTES

PRACTICE SECTION

EASY

Build confidence with one method at a time. Focus on exact output and case sensitivity.

E13 Count spaces

FOCUS: count()

Count the number of ordinary space characters in the sentence. Do not count letters or punctuation.

TEST CASE

```
text = "Python is fun"
```

EXPECTED RESULT

```
2
```

MY PLAN / NOTES

E14 Find the at-sign

FOCUS: find()

Find the index of the first "@" character in the email-like string.

TEST CASE

```
text = "help@example.com"
```

EXPECTED RESULT

```
4
```

MY PLAN / NOTES

E15 Capitalize a sentence-like string

FOCUS: capitalize()

Apply capitalize() to the whole string. Do not capitalize every word separately.

TEST CASE

```
text = "hELLO WORLD"
```

EXPECTED RESULT

```
Hello world
```

MY PLAN / NOTES

PRACTICE SECTION

MEDIUM

Combine a method with simple decisions, multiple occurrences, or a carefully chosen test case.

M01 Classify a text file

FOCUS: endswith()

Write a program that prints True if the given filename ends with either ".csv" or ".txt"; otherwise print False. Treat the suffix check as case-sensitive.

TEST CASE

```
filename = "data.csv"
```

EXPECTED RESULT

```
True
```

MY PLAN / NOTES

M02 Normalize a single-word name

FOCUS: capitalize()

A user enters a single-word first name with inconsistent casing. Print the normalized value using capitalize(). Do not assume this method title-cases multiple words.

TEST CASE

```
name = "aNANYA"
```

EXPECTED RESULT

```
Ananya
```

MY PLAN / NOTES

M03 Replace every matching phrase

FOCUS: replace()

Replace every occurrence of "very late" with "delayed". The replacement must affect all matching occurrences, not just the first one.

TEST CASE

```
text = "The bus is very late; the train is very late"
```

EXPECTED RESULT

```
The bus is delayed; the train is delayed
```

MY PLAN / NOTES

PRACTICE SECTION

MEDIUM

Combine a method with simple decisions, multiple occurrences, or a carefully chosen test case.

M04 Report a search result clearly

FOCUS: find()

Search for "error". If find() returns -1, print "Not found"; otherwise print the index. Match capitalization exactly.

TEST CASE

```
text = "No issue detected"
```

EXPECTED RESULT

```
Not found
```

MY PLAN / NOTES

M05 Count non-overlapping matches

FOCUS: count()

Count occurrences of the substring "ha". Use count() and report its result; do not count overlapping matches separately.

TEST CASE

```
text = "hahaha"
```

EXPECTED RESULT

```
3
```

MY PLAN / NOTES

M06 Replace only the first match

FOCUS: replace()

Replace only the first occurrence of "2024" with "2025". Leave any later occurrences unchanged.

TEST CASE

```
text = "Plan 2024; archive 2024"
```

EXPECTED RESULT

```
Plan 2025; archive 2024
```

MY PLAN / NOTES

PRACTICE SECTION

MEDIUM

Combine a method with simple decisions, multiple occurrences, or a carefully chosen test case.

M07 Search within a range

FOCUS: find()

Use find() with start and end positions to search for "coding" only within the specified slice of the text. Print the returned index.

TEST CASE

```
text = "Python makes coding easier"; search range = [7, 19)
```

EXPECTED RESULT

```
13
```

MY PLAN / NOTES

M08 Reject misleading filenames

FOCUS: endswith()

A file should be accepted as a PDF only if its name ends exactly with ".pdf". Do not accept a filename merely because ".pdf" appears somewhere in the middle.

TEST CASE

```
filename = "report.pdf.backup"
```

EXPECTED RESULT

```
False
```

MY PLAN / NOTES

M09 Count repeated character pairs

FOCUS: count()

Count occurrences of "aa" using count(). Remember that count() does not count overlapping matches.

TEST CASE

```
text = "aaaaa"
```

EXPECTED RESULT

```
2
```

MY PLAN / NOTES

PRACTICE SECTION

MEDIUM

Combine a method with simple decisions, multiple occurrences, or a carefully chosen test case.

M10 Replace a separator

FOCUS: replace()

Replace every hyphen with a space so a hyphen-separated label becomes readable. Preserve the order of all words.

TEST CASE

```
text = "first-name-last-name"
```

EXPECTED RESULT

```
first name last name
```

MY PLAN / NOTES

M11 Count case variants separately

FOCUS: count()

Print three separate counts: occurrences of "error", "Error", and "ERROR". The method is case-sensitive.

TEST CASE

```
text = "Error error ERROR"
```

EXPECTED RESULT

```
1, 1, 1
```

MY PLAN / NOTES

M12 Update a domain in a message

FOCUS: replace()

Replace every exact occurrence of "example.com" with "example.org". Keep names and punctuation unchanged.

TEST CASE

```
text = "help@example.com, sales@example.com"
```

EXPECTED RESULT

```
help@example.org, sales@example.org
```

MY PLAN / NOTES

PRACTICE SECTION

MEDIUM

Combine a method with simple decisions, multiple occurrences, or a carefully chosen test case.

M13 Recognize an exact lowercase suffix

FOCUS: endswith()

Determine whether this filename ends with the exact lowercase suffix ".jpg". Do not convert the input to another case.

TEST CASE

```
filename = "holiday.JPG"
```

EXPECTED RESULT

```
False
```

MY PLAN / NOTES

M14 Count vowels with count()

FOCUS: count()

Using count() for the five lowercase vowels, calculate the total number of lowercase vowels in the given word. You may add the five counts.

TEST CASE

```
text = "education"
```

EXPECTED RESULT

```
5
```

MY PLAN / NOTES

M15 Check an academic email suffix

FOCUS: endswith()

Return True if the string ends with either ".edu" or ".ac.in". This is a simple suffix exercise, not full email validation.

TEST CASE

```
email = "student@college.ac.in"
```

EXPECTED RESULT

```
True
```

MY PLAN / NOTES

PRACTICE SECTION

HARD

Reason about method behavior, edge cases, search positions, and interactions between multiple operations.

H01 Redact only the first matching ID

FOCUS: replace()

Replace only the first occurrence of the provided account ID with "[REDACTED]". Later copies must remain unchanged. Use replace() with its count argument.

TEST CASE

```
text = "ID 71A9 was logged; 71A9 is pending"; target = "71A9"
```

EXPECTED RESULT

```
ID [REDACTED] was logged; 71A9 is pending
```

MY PLAN / NOTES

H02 Classify multiple allowed suffixes

FOCUS: endswith()

Accept filenames ending in ".py", ".pyw", or ".ipynb". Reject all other endings. Use a tuple of suffixes rather than writing three separate checks.

TEST CASE

```
filename = "analysis.ipynb"
```

EXPECTED RESULT

```
Accepted
```

MY PLAN / NOTES

H03 Find overlapping matches

FOCUS: find()

Find the first occurrence of "ana", then find the next occurrence starting one character after the first match begins. Print both positions. The matches may overlap.

TEST CASE

```
text = "banana"
```

EXPECTED RESULT

```
1, 3
```

MY PLAN / NOTES

PRACTICE SECTION

HARD

Reason about method behavior, edge cases, search positions, and interactions between multiple operations.

H04 Avoid a chained-replacement trap

FOCUS: replace()

The text is "catdog". Your goal is to change the original "cat" to "dog" and the original "dog" to "fox", producing "dogfox". Explain why a direct left-to-right chain can fail, then implement a safe approach using replace() and a temporary placeholder.

TEST CASE
```
text = "catdog"
```

EXPECTED RESULT
```
dogfox
```

MY PLAN / NOTES

H05 Explain capitalization side effects

FOCUS: capitalize()

Apply capitalize() to the input and print the result. Then add a brief comment explaining why this operation would be unsuitable if the goal were to preserve the original capitalization of every other character.

TEST CASE
```
text = "mIXed CASE"
```

EXPECTED RESULT
```
Mixed case
```

MY PLAN / NOTES

H06 Reject a disguised executable

FOCUS: endswith()

A user uploads a filename. Accept it only when it ends in ".pdf" or ".png". Explain through test cases why checking for the substring ".pdf" anywhere in the name is weaker than checking the suffix.

TEST CASE
```
filename = "invoice.pdf.exe"
```

EXPECTED RESULT
```
Rejected
```

MY PLAN / NOTES

PRACTICE SECTION

HARD

Reason about method behavior, edge cases, search positions, and interactions between multiple operations.

H07 Find the second occurrence

FOCUS: find()

Find the first occurrence of "error" and then locate the next occurrence by starting the second search after the first match. If there is no second occurrence, report -1.

TEST CASE

```
text = "error at line 5; error at line 9"
```

EXPECTED RESULT

```
17
```

MY PLAN / NOTES

H08 Count case-sensitive log labels

FOCUS: count()

Count exact uppercase "ERROR" labels and exact lowercase "error" strings separately. Do not combine the two counts.

TEST CASE

```
text = "ERROR: disk; error: ignored; ERROR: retry"
```

EXPECTED RESULT

```
2 uppercase, 1 lowercase
```

MY PLAN / NOTES

H09 Replace a placeholder using a variable

FOCUS: replace()

Read a name from the user and replace every exact occurrence of "<NAME>" in a template with that name. The template may contain the placeholder more than once.

TEST CASE

```
template = "Welcome, <NAME>. Your profile is <NAME>."; name = "Mira"
```

EXPECTED RESULT

```
Welcome, Mira. Your profile is Mira.
```

MY PLAN / NOTES

PRACTICE SECTION

HARD

Reason about method behavior, edge cases, search positions, and interactions between multiple operations.

H10 Understand find() at index zero

FOCUS: find()

The search target appears at the start of the string. Write a correct condition that checks whether the target exists. Do not use the returned index as a simple True/False value.

TEST CASE

```
text = "Python is ready"; target = "Python"
```

EXPECTED RESULT

```
Found at index 0
```

MY PLAN / NOTES

H11 Reduce repeated spaces carefully

FOCUS: replace()

A single replace(" ", " ") call does not always reduce every run of repeated spaces to one space. Write a solution that repeatedly applies the replacement until no double spaces remain, then print the cleaned text.

TEST CASE

```
text = "Python string practice"
```

EXPECTED RESULT

```
Python string practice
```

MY PLAN / NOTES

H12 Distinguish a substring from a whole word

FOCUS: count()

The text contains "error" and "errors". First report count("error"). Then explain in a code comment why that number is not necessarily the count of the whole word "error" only. This task assesses the method's semantics; do not silently treat count() as a word-boundary search.

TEST CASE

```
text = "error errors error"
```

EXPECTED RESULT

```
3 substring matches
```

MY PLAN / NOTES

PRACTICE SECTION

HARD

Reason about method behavior, edge cases, search positions, and interactions between multiple operations.

H13 Use endswith() with a file policy

FOCUS: endswith()

A system accepts a file only when its final suffix is exactly ".csv" or ".json". Write a check and test both a valid filename and a misleading filename that contains an allowed suffix earlier in its name.

TEST CASE

```
filename = "backup.csv.old"
```

EXPECTED RESULT

```
Rejected
```

MY PLAN / NOTES

H14 Store the result of replace()

FOCUS: replace()

Predict what is printed when replace() is called but its returned value is not assigned. Then correct the code so the updated string is printed.

TEST CASE

```
text = "mississippi"; text.replace("s", "S"); print(text)
```

EXPECTED RESULT

```
First print: mississippi
```

MY PLAN / NOTES

H15 Compare overlapping and counted matches

FOCUS: count()

For the text "abababa", count occurrences of "aba" with count(). In a comment, state how many starting positions could produce overlapping matches and why count() returns a smaller number.

TEST CASE

```
text = "abababa"; target = "aba"
```

EXPECTED RESULT

```
2 non-overlapping matches
```

MY PLAN / NOTES

PRACTICE SECTION

REAL-WORLD & TRICKY

Small realistic requirements that combine several of the five methods. Define edge-case behavior explicitly.

R01 Upload filename policy

FOCUS: endswith()

Build a small file-type checker for a fictional upload form. Accept only names ending in ".pdf", ".png", ".jpg", or ".jpeg". Print "Accepted" or "Rejected". The rule is case-sensitive and checks only the final suffix; it is not a security-grade file validator.

TEST CASE

```
Test: "assignment.pdf", "photo.jpeg", "report.pdf.exe", "photo.JPG"
```

EXPECTED RESULT

```
Accepted, Accepted, Rejected, Rejected
```

MY PLAN / NOTES

R02 Summarize an application log

FOCUS: count(), find()

Given a log string, count exact uppercase "ERROR" labels and find the index of the first one. If no uppercase label exists, print a clear no-error message instead of reporting index -1 as if it were a real location.

TEST CASE

```
log = "INFO: start | ERROR: timeout | INFO: retry | ERROR: failed"
```

EXPECTED RESULT

```
Count: 2; first index: 14
```

MY PLAN / NOTES

R03 Redact repeated customer IDs

FOCUS: replace()

A support message may mention the same customer ID multiple times. Replace every exact occurrence of the supplied ID with "[REDACTED]". Leave similar-but-not-identical IDs untouched, and store the returned string.

TEST CASE

```
message = "Customer C-104 contacted us. Verify C-104 before refunding."; customer_id = "C-104"
```

EXPECTED RESULT

```
Customer [REDACTED] contacted us. Verify [REDACTED] before refunding.
```

MY PLAN / NOTES

PRACTICE SECTION

REAL-WORLD & TRICKY

Small realistic requirements that combine several of the five methods. Define edge-case behavior explicitly.

R04 Normalize a display name

FOCUS: capitalize()

A registration form accepts a single-word display name. Normalize inconsistent casing with capitalize(). If the input is empty, print "Name required" instead. Do not claim this method correctly formats arbitrary multi-word names.

TEST CASE

```
Test: "sHReyASH" and ""
```

EXPECTED RESULT

```
Shreyash; Name required
```

MY PLAN / NOTES

R05 Audit a promotional code

FOCUS: count(), find(), replace()

Given an order note and a promotional code, count exact occurrences and find the first position. If the code appears at least once, replace every occurrence with "[CODE APPLIED]"; otherwise leave the note unchanged and print "Code not found". Match case exactly.

TEST CASE

```
note = "SAVE10 applied; customer asked about SAVE10"; code = "SAVE10"
```

EXPECTED RESULT

```
Count: 2; first index: 0; both occurrences replaced
```

MY PLAN / NOTES

Progress Tracker

Use this page to track independent practice, not just whether code ran once. Revisit anything solved with hints.

Easy

☐ E01	☐ E02	☐ E03	☐ E04	☐ E05
☐ E06	☐ E07	☐ E08	☐ E09	☐ E10
☐ E11	☐ E12	☐ E13	☐ E14	☐ E15

Medium

☐ M01	☐ M02	☐ M03	☐ M04	☐ M05
☐ M06	☐ M07	☐ M08	☐ M09	☐ M10
☐ M11	☐ M12	☐ M13	☐ M14	☐ M15

Hard

☐ H01	☐ H02	☐ H03	☐ H04	☐ H05
☐ H06	☐ H07	☐ H08	☐ H09	☐ H10
☐ H11	☐ H12	☐ H13	☐ H14	☐ H15

Real-world / tricky

☐ R01	☐ R02	☐ R03	☐ R04	☐ R05

A question is complete when you can: (1) explain the method behavior, (2) pass the sample and your own edge case, and (3) write the solution again without looking at your previous code.